Is a fixed-size memory enough to translate?

Pure RNNs vs. one Kimi Linear block on byte-level English→Finnish

Toni Huhtiniemi

October 2026

Abstract

We train four small translation models (2.5–4.6M parameters) on the same data, with the same optimizer, schedule and seed. Three are *pure* RNN encoder-decoders: the whole English sentence is squeezed into the final RNN state, which becomes the decoder's starting state, and the decoder never sees the source again (GRU, LSTM, and a 2-layer minGRU). The fourth is a decoder-only Kimi Linear block (3 Kimi Delta Attention layers + 1 MLA attention layer) that can look back at the source. The Kimi block reaches **0.96 bits/byte**, against **1.44** for the best RNN (LSTM), while having fewer parameters. It uses about twice the compute, but it is still far ahead when compute or wall-clock time is matched. Among the RNNs, the model with the best-protected memory wins: LSTM > GRU > minGRU.

1 Question and setup

Early neural translation (Sutskever et al., 2014) encoded a sentence into one fixed-size vector. We ask how much this bottleneck costs, compared with a modern hybrid model that can attend to the source, at roughly the same model size.

Data. OPUS-100 English–Finnish (1M sentence pairs). We keep pairs with at most 128 bytes on each side (918k pairs; 33.9 MB English and 31.7 MB Finnish per epoch). The same 2,000 random pairs are held out for validation for every model. Models read and write raw UTF-8 bytes (256 bytes + 3 special tokens), so the loss is in *bits per byte* of Finnish text, and is directly comparable across models.

Models. All RNNs use width 512 and one layer per side unless noted.

- **GRU**: encoder GRU → final state h (512 numbers) → initial state of decoder GRU. The decoder input is only the previous byte.
- **LSTM**: the same with LSTM cells; both h and c are passed (1,024 numbers). This is a minimal Sutskever et al. model (1 layer, source not reversed).
- **minGRU**: 2 stacked minGRU layers per side (Feng et al., 2024). Its gates depend only on the input, so the recurrence is linear and trains with a parallel scan. Each layer's final state is passed (1,024 numbers).
- **Kimi**: one Kimi Linear block (Kimi Team, 2025), decoder-only, width 256. Input is [English bytes, separator, Finnish bytes]; loss only on the Finnish part. Three KDA layers (gated delta rule with per-channel decay, short convolutions, 4 heads of 64, state 4×64×64) and one MLA layer (latent attention, no positional encoding), each followed by a SwiGLU MLP. Simplified: dense MLP instead of mixture-of-experts, plain PyTorch implementation. *There is no bottleneck*: MLA sees every source byte.

Training. 5 epochs, batch 256 (17,895 steps), learning rate $2{\cdot}10^{-3}$ with 500 warm-up steps and cosine decay to 10%. Muon for hidden weight matrices, AdamW for embeddings, output layer and biases; weight decay 0.01, gradient clipping 1.0, fp16. One NVIDIA RTX 2080 Super (8 GB). Translations use beam search (width 5).

2 Results

Model	Params	State passed	Val bits/byte	Train bits/byte	Compute*	Time	Test score
GRU	3.55M	512	1.53	1.42	3.2 PFLOP	11 min	3.5 / 25
LSTM	4.60M	1,024	1.44	1.30	4.3 PFLOP	14 min	6 / 25
minGRU (2 layers)	2.50M	1,024	1.68	1.60	2.2 PFLOP	22 min	3 / 25
Kimi block	3.43M	none (attends)	**0.96**	0.87	7.0 PFLOP	93 min	**19.5 / 25**

Table 1: Final numbers. Train loss is the average over the last 100 steps. Test score: five new everyday sentences (Table 2), each scored 0–5 by hand for meaning and grammar (one judge, not a native speaker). *Estimated useful training FLOPs: 2 per multiply-add for every weight matrix applied to every byte it touches (RNN encoder on English bytes, decoder and output layer on Finnish bytes; Kimi: all weights on all bytes), plus KDA state updates and causal attention; backward pass = 2× forward. Padding is not counted.

3 Main observations

1. **The bottleneck is the main problem.** The Kimi block's loss is 33% lower than the best RNN's, with fewer parameters than the LSTM. It passes the RNNs' *final* loss after only ∼2,000 of 17,895 steps.

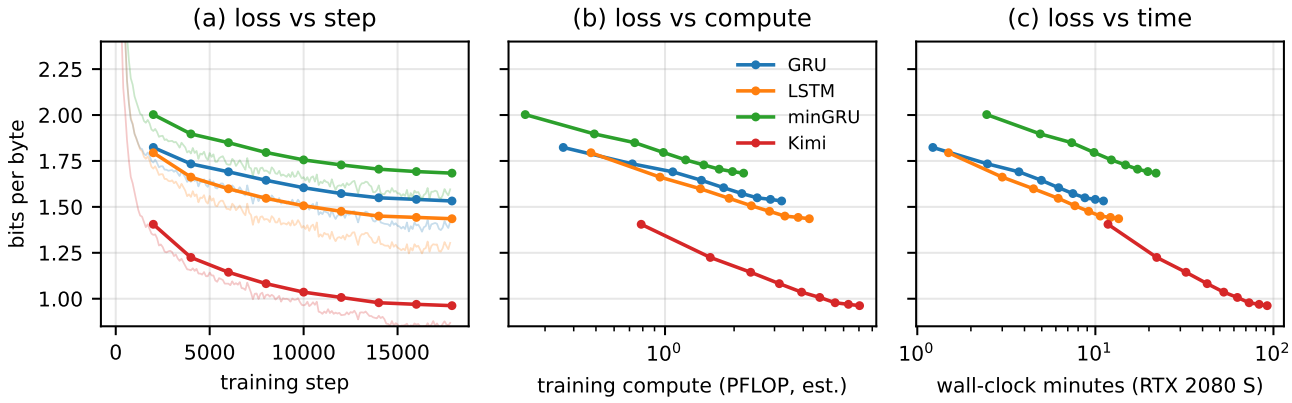


Figure 1: Validation loss (bits per Finnish byte, lower is better). (a) per training step; faint lines are training loss. (b) against estimated training compute. (c) against measured wall-clock time (evaluation excluded). The Kimi block is better at every step, every compute budget and every time budget.

English	GRU	LSTM	minGRU	Kimi
My sister bought a new car last week.	Sisälläni on paljon vaihtoehtoja. *("inside me are many options")*	Siskoni kuoli viime viikolla. *("my sister died last week")*	Minulla on oikeassa. *(broken)*	Siskoni osti uuden auton. *(correct, drops "last week")*
Can you help me find the train station?	Autatko meidän kanssamme? *("help with us?")*	Voitteko auttaa meitä? *("can you help us?")*	Auttaisitko mitään? *("help anything?")*	Voitko auttaa minua etsimään junaa? *("... look for the train")*
The children are playing in the garden.	Lapset ovat valmiina. *("are ready")*	Lapset pitävät sisään. *(broken)*	Lapset ovat tulevaisuudesta. *("are from the future")*	Lapset pelaavat puutarhassa. *(good; "leikkivät" is better)*
We should protect the environment for future generations.	Meidän pitäisi olla erittäin tärkeää. *(broken)*	Meidän pitäisi käyttää vähintään yhteistyötä vastaan. *(word salad)*	Meidän pitäisi olla täällä. *("we should be here")*	Meidän pitäisi suojella ympäristöä. *(correct, drops the end)*

Table 2: Example translations (4 of the 5 test sentences; none of them appears in the training data).

2. **It is not just extra compute.** Decoder-only models run all weights over both sentences, so Kimi used 2.2× the GRU's compute. But at the GRU's full budget (3.2 PFLOP), Kimi was already at 1.08 bits/byte vs. 1.53 (Fig. 1b). Even by wall-clock time, with our slow plain-PyTorch kernels, Kimi reached 1.40 bits/byte at 12 minutes, better than the GRU after its full 11-minute run (Fig. 1c).
3. **Among RNNs, protected memory wins: LSTM > GRU > minGRU.** The LSTM's separate cell state keeps the source longer than the GRU's single state. minGRU is worst even with two layers: it has no hidden-to-hidden matrix, so each unit is just a gated running average of its input and cannot combine information over time.
4. **Different failure styles.** All RNNs keep the first words ("Siskoni", "Meidän pitäisi", "Lapset") and lose the content words (car, station, garden, environment). The LSTM writes fluent Finnish that says something else ("my sister died"); minGRU writes broken Finnish. Kimi keeps the content words; its errors are word choice ("pelata" vs. "leikkiä") and dropped final phrases.
5. **No model is overfitting much.** Train/validation gaps are about 0.1 bits/byte and all curves are still going down, so longer training would help all of them.

4 Limitations

One seed per model; small models and data; validation is a random split of the OPUS training file, not the official test set; the test score uses only five sentences and one non-native judge. Kimi has width 256 vs. 512 for the RNNs and is a simplified re-implementation. A cleaner test of attention alone would add attention (Bahdanau et al., 2014) to the GRU and keep everything else the same.

References. Bahdanau, Cho, Bengio (2014). Neural machine translation by jointly learning to align and translate. Cho et al. (2014). Learning phrase representations using RNN encoder-decoder. Sutskever, Vinyals, Le (2014). Sequence to sequence learning with neural networks. Feng et al. (2024). Were RNNs all we needed? Kimi Team (2025). Kimi Linear: an expressive, efficient attention architecture. Zhang et al. (2020). Improving massively multilingual NMT and zero-shot translation (OPUS-100). Jordan et al. (2024). Muon: an optimizer for hidden layers.